Phase 0 · Production document

Technical Design Document

How the game is built: project setup, module design, events, data formats, Blender conventions, saves, AI, rendering, audio, performance and deployment.

Stack	TypeScript 7 · Babylon.js 9.31 · Havok 1.3 · Vite 8
Extends	Development Plan, section 3
Version	1.0 · October 2026
Author	Rithy Bondeth

Contents

01 Principles

02 Project Setup

03 Modules

04 Boot and Game Loop

05 Event Catalogue

06 Data Formats

07 Blender and glTF Conventions

08 Asset Pipeline

09 Scripted Sequences

10 Perception and AI

11 Rendering

12 Audio

13 Saves and Save Codes

14 Performance and Memory

15 Testing, Standards and Deployment

01 Principles

Principle	In practice
Systems are generic; episodes are data	No episode-specific code in `systems/`. Episodes provide data files, Blender scenes and a small number of sequence scripts.
Loose coupling	Systems communicate through a typed event bus and the flag store, never by importing each other.
Blender is the level editor	Placement of triggers, items, patrols and zones lives in Blender, exported as glTF extras.
Use the engine	Physics, audio, picking, GUI and post-processing come from Babylon.js; we write glue and game logic, not engine features.
Testable logic	Game rules (inventory, flags, saves, AI decisions) are plain TypeScript that can be unit-tested without a renderer.

API names below match Babylon.js 9. Check the current documentation when implementing, since minor APIs change between releases.

02 Project Setup

2.1 Dependencies (pinned exact versions)

```
"dependencies": {
  "@babylonjs/core":    "9.31.0",
  "@babylonjs/loaders": "9.31.0",
  "@babylonjs/gui":     "9.31.0",
  "@babylonjs/havok":   "1.3.14",
  "idb-keyval":         "(latest at setup)"
},
"devDependencies": {
  "typescript": "7.0.2",
  "vite":       "8.3.4",
  "vitest":     "(latest at setup)",
  "@playwright/test":   "(latest at setup)",
  "@gltf-transform/cli": "(latest at setup)",
  "eslint", "prettier"
}
```

2.2 Scripts

Command	Does
`npm run dev`	Vite dev server with hot reload
`npm run build`	Type-check and production build to `dist/`
`npm run test`	Vitest unit tests
`npm run e2e`	Playwright smoke test: the game boots to the menu in WebGPU and WebGL2 modes
`npm run assets`	Optimise everything in `art-export/` into `public/assets/` (section 8)
`npm run docs`	Rebuild the design PDFs (`tools/build-docs.sh`)

2.3 Source layout

```
src/
├── main.ts                Boot (section 4)
├── core/
│   ├── Game.ts            Owns engine, scene, loop, top-level state
│   ├── EventBus.ts        Typed publish/subscribe (section 5)
│   ├── Flags.ts           Story flag store
│   ├── AreaLoader.ts      Loads/unloads area asset containers
│   ├── LevelBuilder.ts    Turns glTF extras into game objects (section 7)
│   ├── Time.ts            Game clock, pause, time scale
│   └── Localization.ts    String tables (en, km)
├── player/                Controller, CameraRig, Interactor, Hands
├── systems/               Inventory, Sketchbook, Lamp, Matches, Upkeep,
│                          Hiding, Checkpoints, Saves, Settings, NightCount
├── ai/                    Perception, NoiseSystem, StateMachine, entities/
├── sequences/             Sequencer runtime + shared helper actions
├── audio/                 AudioSystem, Buses, AmbienceZones, Cues
├── render/                RenderSetup, PostFx, QualityPresets, materials/
├── ui/                    diegetic/ (sketchbook pages, gauges), html/ (menus)
├── episodes/ep1/          index.ts, sequences.ts, data/*.json
└── data/                  items.json, notes.json, strings/en.json, strings/km.json
```

03 Modules

Module	Responsibility	Listens / emits
Game	Creates the engine and scene, runs the loop, switches between menu, episode and pause states.	Emits `game:*`
AreaLoader	Loads an area's `.glb` into an asset container, adds or removes it from the scene, disposes it on unload.	Emits `area:loaded/unloaded`
LevelBuilder	Reads node names and extras, creates triggers, interactables, spawn points, patrols, hiding spots, audio zones and lights.	Emits `level:built`
Flags	Key-value store for story state (boolean, number, string). Serialised by Saves.	Emits `flag:changed`
PlayerController	Havok character controller: walk, crouch, crawl, climb stairs; head-bob; footstep timing.	Emits `player:step`, `noise:made`
Interactor	Ray from the screen centre (max 1.8 m); highlights and activates interactables.	Emits `interact:*`
Inventory	Items, counts, combining (key fragments → key).	Emits `inventory:changed`
Sketchbook	Pages and entries unlocked by flags; renders pages to dynamic textures; map and night count.	Listens to `flag:changed`
Lamp	States open / shuttered / out; oil level and drain rate; flicker; light radius.	Emits `lamp:*`
Upkeep	Generic decaying timers (incense) with warning stages and consequences.	Emits `upkeep:*`
Perception / NoiseSystem	Sight checks and noise events for all AI.	Listens to `noise:made`, `lamp:*`
Entities	Per-entity state machines built from entity data.	Emits `entity:*`, `player:caught`
Sequencer	Runs async scripted sequences; can lock input and the camera.	Emits `sequence:*`
Checkpoints / Saves	Snapshot and restore state; IndexedDB slots; save codes.	Listens to `player:caught`
AudioSystem	Engine creation and unlock, buses, spatial sounds, ambience zones, reverb.	Listens to most events
RenderSetup / PostFx	Engine options, pipeline, quality presets, colour grading per episode.	Listens to `settings:changed`

04 Boot and Game Loop

```
1. main.ts
   ├─ if (await WebGPUEngine.IsSupportedAsync) → new WebGPUEngine(canvas); await initAsync()
   │  else → new Engine(canvas, true)                    // WebGL2
   ├─ show HTML loading screen
   ├─ load Havok WASM: await HavokPhysics() → new HavokPlugin(true, havok)
   ├─ scene.enablePhysics(new Vector3(0, -9.81, 0), plugin)
   ├─ load settings + save index from IndexedDB
   ├─ load shared assets (lamp, hands, satchel, sketchbook, UI sounds)
   └─ main menu (HTML) — "Start" click also unlocks audio
2. Start episode
   ├─ AreaLoader.load(first area) → LevelBuilder.build()
   ├─ restore checkpoint or start at beat P01
   └─ engine.runRenderLoop(() => { Time.tick(); systems.update(dt); scene.render(); })
```

Update order each frame	Reason
1. Input → PlayerController	Movement before anything reads the player position
2. Interactor, Lamp, Upkeep	Player-driven state
3. NoiseSystem, Perception, Entities	AI reacts to this frame's player state
4. Sequencer	Scripts can override everything above
5. Audio listener, PostFx parameters	Presentation last

05 Event Catalogue

```
type GameEvents = {
  "flag:changed":      { key: string; value: FlagValue };
  "area:loaded":       { areaId: string };
  "interact:focus":    { id: string } ;  "interact:use": { id: string };
  "inventory:changed": { itemId: string; count: number };
  "lamp:state":        { state: "open" | "shuttered" | "out" };
  "lamp:oil":          { level: number };            // 0..1
  "upkeep:warning":    { id: string; stage: 1 | 2 };
  "upkeep:expired":    { id: string };
  "noise:made":        { pos: Vector3; loudness: number; source: string };
  "entity:state":      { id: string; state: AiState };
  "player:caught":     { by: string };
  "checkpoint:reached":{ id: string };
  "sequence:start":    { id: string };  "sequence:end": { id: string };
  "choice:made":       { id: string; option: string };
  "settings:changed":  { key: string };
};
```

The bus is a small typed wrapper (`on` , `off` , `emit`). Handlers must not throw; errors are caught and logged with the event name.

06 Data Formats

6.1 Items, notes and strings

```
interface ItemDef {
  id: string;                    // "oil_bottle", "key_fragment"
  nameKey: string;               // string-table key
  model: string;                 // "shared/oil_bottle.glb"
  stack: number;                 // max carried, 1 for unique items
  combine?: { with: string; count: number; result: string };
  onUse?: string;                // action id, e.g. "lamp.refill"
}
interface NoteDef {
  id: string;                    // "lore.ep1.L1"
  titleKey: string; bodyKey: string;
  lore: boolean;                 // counts toward Knowledge
  confirmsChant?: string[];      // Episode 5 verse ids it confirms
}
// data/strings/en.json : { "note.ep1.L1.body": "Kanha used to make…", … }
// data/strings/km.json : same keys, Khmer text
```

6.2 Entities

```
interface EntityDef {
  id: string;                    // "ep1_roamer"
  model: string;
  speed: { patrol: number; chase: number };   // m/s
  hearing: { normal: number; crouched: number }; // radius in m
  sight?: { lampRange: number; darkRange: number; fovDeg: number };
  patrol: string;                // Blender patrol path id
  catchRadius: number;
}
```

6.3 Flags

- Keys: `ep<n>.<topic>.<state>` for story, `lore.ep<n>.<id>` for notes, `series.<counter>` for cross-episode values.
- Cross-episode counters: `series.respect` (0–9), `series.vuthy.saved`, `series.vuthy.forgiven`, `series.mother.freed`, `series.chantErrors`.
- Flags are only ever set through `Flags.set()` so every change emits `flag:changed`.

07 Blender and glTF Conventions

7.1 Object name prefixes

Prefix	Becomes	Custom properties (glTF extras)
`trigger_`	Invisible trigger volume (box)	`beat` (e.g. "A2-01"), `once` (bool), `requires` (flag)
`interact_`	Interactable object	`action` ("pickup", "open", "use"), `item`, `note`
`spawn_`	Player or NPC spawn point	`checkpoint` (e.g. "CP3")
`patrol_<id>_NN`	Ordered patrol nodes	`wait` (seconds)
`hide_`	Hiding spot volume	—
`audiozone_`	Ambience and reverb zone	`ambience`, `reverb`
`light_flame_`	Flickering point light + flame card	`radius`, `intensity`
`col_`	Invisible collision mesh (static physics)	`surface` ("teak", "dirt", "mud", "leaves", "stone", "water")
`nav_`	Navigation mesh source (not rendered)	—

7.2 Asset naming

- Files: `<scope>_<area>_<name>_<variant>.glb`, e.g. `ep1_house_wall_a.glb`, `shared_lamp.glb`.
- Area scenes: `ep1_area_courtyard.glb`, `ep1_area_house.glb`, `ep1_area_underhouse.glb`, `ep1_area_studio.glb`.
- Materials: `M_<name>`; textures: `T_<name>_<map>` where map is `BC`, `N` or `ORM`.

7.3 Export settings

glTF Binary (`.glb`), +Y up, apply modifiers, export custom properties, selected collections only, no cameras or Blender lights (lights are created from `light_` nodes).

08 Asset Pipeline

```
# tools/build-assets (run by: npm run assets) — per file in art-export/
gltf-transform dedup   in.glb  tmp.glb
gltf-transform prune   tmp.glb tmp.glb
gltf-transform weld    tmp.glb tmp.glb
gltf-transform resize  tmp.glb tmp.glb --width 2048 --height 2048   # 1024 for props
gltf-transform etc1s   tmp.glb tmp.glb --slots "{baseColorTexture,emissiveTexture}"
gltf-transform uastc   tmp.glb tmp.glb --slots "{normalTexture,occlusionTexture}"
gltf-transform meshopt tmp.glb public/assets/ep1/out.glb              # or draco
```

KTX2 encoding needs KTX-Software (`toktx`) installed. Check exact flags with `gltf-transform --help` for the installed version.

09 Scripted Sequences

Sequences are async functions given a context of helper actions. They read like the beat sheet.

```
// episodes/ep1/sequences.ts
export async function A2_01_lampsOut(s: SequenceContext) {
  await s.checkpoint("CP2");
  s.lights.extinguishGroup("tent_hanging", { stagger: 0.15 });
  s.audio.play("gust_cold", { at: "tent_center" });
  await s.wait(1.2);
  s.audio.play("footsteps_wet_above", { at: "house_hall" });
  s.flags.set("ep1.act", 2);
}

export async function A3_07_ritual(s: SequenceContext) {
  await s.checkpoint("CP6");
  s.player.lockMovement();
  await s.waitFor(() => s.inventory.used("bronze_token", "burner_main"));
  await s.waitFor(() => s.inventory.used("sanctified_match", "burner_main"));
  const ok = await s.holdWithoutTurning({ seconds: 25, maxAngleDeg: 100,
                                          during: () => s.presence.approach() });
  if (!ok) return s.fail("CP6");
  s.flags.set("ep1.seal1.broken", true);
}
```

10 Perception and AI

10.1 Perception

- **Hearing:** every `noise:made` event has a loudness (0–1). An entity hears it if distance ≤ its hearing radius × loudness, with walls reducing loudness by 50% (one raycast).
- **Sight:** inside the field of view and within range, then one line-of-sight raycast to the player's head. The range depends on the lamp: open lamp uses `lampRange`; shuttered or out uses `darkRange`.
- Checks run at 10 Hz, not every frame.

10.2 State machine

State	Behaviour	Transitions
Patrol	Follow patrol nodes, waiting where set	Heard noise → Investigate · saw player → Chase
Investigate	Move to the noise position; look around for 4 s	Saw player → Chase · nothing → Return
Chase	Move to the player at chase speed	Within catch radius → Catch · lost sight 5 s → Search
Search	Visit the last known position and two nearby nodes	Saw player → Chase · timeout → Return
Return	Path back to the nearest patrol node	Arrived → Patrol
Catch	Emit `player:caught`; play the catch sequence	Checkpoint restore
Scripted	Controlled entirely by a sequence (walker, presence)	Sequence ends

Pathing uses a navigation mesh baked from `nav_` geometry with Babylon's Recast-based navigation plugin.

11 Rendering

Setting	Low	Medium	High
Render scale	0.75	1.0	1.0
Lamp shadow map	512	1024	2048
Unshadowed flame lights	8	16	32
Grain, vignette, grading	On	On	On
Bloom	Off	On	On
Chromatic aberration	Off	On	On
Volumetric fog (Ep 4)	Fog cards	Fog cards	Volumetric

- Post-processing through `DefaultRenderingPipeline`; per-episode colour grading lookup tables.
- Static architecture uses baked lightmaps; meshes in each area are merged by material where possible.
- The first launch runs a 3-second benchmark scene and chooses a preset; the player can change it.
- Diegetic pages use `DynamicTexture` or `AdvancedDynamicTexture.CreateForMesh` on the sketchbook mesh.

12 Audio

Bus	Contents	Notes
Master	Everything	Master volume setting
Ambience	Rain, insects, wind, room tones	Streamed; crossfaded by audio zones
SFX	Footsteps, objects, lamp, matches	Spatial; surface-dependent footsteps
Threat	Entity sounds, scares	Spatial; never ducked
Voice	Dialogue, Sok's recording	Ducks ambience by −6 dB
Music	Stingers and drones	Streamed
UI	Satchel, pages, menus	Non-spatial

- Audio Engine v2 (`CreateAudioEngineAsync`); the engine is unlocked on the first user click (browser autoplay rules).
- Spatial sounds use HRTF panning for threat sounds (positional accuracy matters) and equal-power panning for minor sounds to save CPU.
- Reverb per audio zone: small wooden room, open courtyard, under-house, large stone hall.
- Formats: Ogg Opus as the primary format, with an AAC fallback for browsers that need it.

13 Saves and Save Codes

```
// IndexedDB key "save.slot1"
{
  "saveVersion": 1,
  "episode": 1,
  "checkpoint": "CP4",
  "clock": "02:00",
  "flags": { "ep1.betel.done": true, "ep1.betel.firstTry": true, "lore.ep1.L1": true, … },
  "inventory": { "oil_bottle": 1, "matches": 7, "sanctified_match": 3, "key_fragment": 1 },
  "lamp": { "oil": 0.64, "state": "open" },
  "playTimeSec": 4230,
  "savedAt": "2026-10-20T14:03:00Z"
}
```

- **Save code** = `"BS1." + base64url(deflate(JSON)) + "." + checksum`. It carries flags and series counters between devices and into the next episode. It is tamper-evident, not secret.
- **Migrations:** `migrations[n]` converts version *n* to *n+1*; old saves are upgraded on load.
- **Checkpoints** store the same structure in memory; reaching a checkpoint also writes slot autosave.

14 Performance and Memory

Rule	Detail
Budgets	As in the Development Plan: 120 draw calls, 150k triangles, ~256 MB textures per area, 85 MB initial download.
Area unload	Dispose the asset container (meshes, materials, textures), stop and dispose area sounds, clear entity references; verify counts in the Inspector return to baseline.
Allocation	No new objects in per-frame code paths (reuse vectors); pool frequently spawned objects.
Profiling routine	Weekly on both reference machines: FPS, draw calls, triangles, JS heap, GPU memory; record results in a simple log.

15 Testing, Standards and Deployment

Testing

- **Unit tests:** Flags, Inventory (including combining), Upkeep timers, Saves (round trip and migrations), save codes, perception maths, AI transitions.
- **Smoke test:** Playwright boots the build in Chrome with WebGPU on and off and checks the menu appears without console errors.
- **Debug tools** (development builds only): jump to any beat or checkpoint, set flags, refill oil, show perception radii, toggle the Babylon Inspector.

Code standards

- TypeScript strict mode; no `any` without a comment explaining why.
- One class or module per file; file names match the export (`Lamp.ts`).
- Beat IDs in comments next to the code that implements them (`// A2-06`).
- Small commits with clear messages; a branch per feature.

Deployment

- Static hosting. Serve `.wasm` as `application/wasm` , `.ktx2` as `image/ktx2` , and enable Brotli or gzip compression.
- Hashed file names for long-term caching; `index.html` never cached.
- Check each host's per-file size limit against the largest `.glb` .